Verifying Trading Strategies

Giolit Trading Risk Verifier 1.0.0 — Tutorial 1

Contents

1. What the verifier does, and what it does not do . . . 1
2. Installing and the license . . . 2
3. The three ways to verify a strategy . . . 2
4. Worked example A: a Python strategy, verified from its source . . . 3
    Step 1 — the binding: what the code means . . . 3
    Step 2 — the specification: the market and the rules that must never break . . . 4
    Step 3 — verify . . . 4
    Step 4 — fix and verify again . . . 4
    What the importer supports . . . 5
5. Worked example B: a strategy written in GTSL . . . 5
6. Worked example C: your risk controls, against any strategy . . . 5
7. Writing good constraints . . . 6
8. Writing honest assumptions . . . 6
9. Reading the results . . . 7
10. When a constraint stays UNDECIDED . . . 7
11. From a result to evidence: headroom, certificates, FIX runs . . . 7
12. Keeping production honest: conformance replay . . . 8
13. Verification as a release gate . . . 8
14. A checklist for every strategy . . . 8

Tutorial 1 of 3 for users of the Giolit Trading Risk Verifier 1.0. The other tutorials are the SDK guide (`../02_sdk_guide`) and the domain primer (`../03_algorithmic_trading_and_verification`). The complete language reference is the Reference Manual (`Reference-Manual.pdf`; `TUTORIAL.md`) in the package.

---

1. What the verifier does, and what it does not do

A back-test answers "what would have happened on *this* historical day". A simulator answers "what happens on the paths I thought of". The verifier answers a different question:

> **Is there any sequence of market moves, fills, partial fills, rejects, cancels, late messages and operator actions — within the assumptions you state — that makes the strategy break one of your rules?**

For each rule (a *constraint*) you get one of these answers:

Answer	What it means for you
PROVEN	No such sequence exists, for any number of orders. The certificate contains the mathematical reason (an inductive invariant).
VIOLATED	Such a sequence exists. You get it event by event, as a FIX 4.4 message log you can replay in your own test harness.
FEASIBLE / INFEASIBLE	For goals ("the parent order can complete"): a run that reaches it, or a proof that none can.
PROVEN*	Proven for runs with at most K orders live at a time (K = 4 by default), not yet beyond.
UNDECIDED	Neither within the time you gave it. The cause is shown; section 10 says what to do.

What it does **not** do:

- It does not predict profit. Against every market your assumptions allow, a strategy that holds inventory can always lose; what *can* be proved is the edge locked in and the worst-case loss (Reference Manual §17).
- It does not check behaviour you did not describe. A proof holds for the model your specification describes — the venue protocol, the assumptions, the ranges. Conformance replay (section 12) checks that production stays inside that model.
- It does not run your code. When it imports your code, it reads the source and builds the model from it; nothing is executed.

2. Installing and the license

```
cd code && ./install.sh           # installs to ~/.local/giolit-trade-verifier and puts trade-verify on
    your PATH
trade-verify --license            # what the installation may do (trial: 1000 runs within 50 days)
trade-verify share/giolit/examples/01_position_limit.gts
```

PDF certificates need LuaLaTeX (`texlive-luatex texlive-latex-extra texlive-fonts-extra` on Debian/Ubuntu). Importing Python strategies needs `python3` on the PATH; Java strategies need a JDK (`java`). A purchased license is applied once:

```
trade-verify --activate=license.json   # asks for the buyer's name, address and password it was issued for
```

3. The three ways to verify a strategy

You have	Use	You write
The strategy's source code (C++, Python or Java)	**Import**	a *binding* (`.gbind`, ~20 lines: which method handles which event, which call sends an order) and a *specification* (`.gts`: assumptions and constraints)
A design, or code you cannot share	**GTSL**	the strategy's rules in GTSL (start from a template: `trade-verify --templates`)

You have	Use	You write
A pre-trade risk layer that must hold for *any* algorithm	**Controls only**	the risk controls (or import them from C++) and the limits they must guarantee

All three end in the same specification file and the same command:

```
trade-verify my_strategy.gts --timeout=120
```

4. Worked example A: a Python strategy, verified from its source

The strategy buys a target quantity in clips, one clip at a time. It is the kind of code a desk writes in an afternoon:

```python
class ClipBuyer:
    def __init__(self, gateway, logger):
        self.gateway = gateway
        self.logger = logger
        self.target = 1000          # shares to buy in all
        self.clip = 100             # shares per order
        self.filled = 0             # shares bought so far
        self.working = False        # an order of ours is at the exchange

    def on_tick(self, tick):
        if self.working or self.filled >= self.target:
            return
        qty = min(self.clip, self.target - self.filled)
        self.logger.info("sending a clip")
        self.gateway.send_buy(qty, tick.bid)
        self.working = True

    def on_trade(self, trade):
        self.filled += trade.qty
        self.working = False

    def on_reject(self):
        self.working = False
```

Step 1 — the binding: what the code means

The binding tells the verifier which method handles which market event, which fields of the event objects the code reads, and which call sends an order. It does not restate any logic.

```
// clip.gbind
binding ClipBuyerPy;
language python;
sources "clip_buyer.py";
ignore logger.info;                      // calls that do not affect trading

strategy ClipBuyer {
    on market_update -> on_tick(tick) {
        tick.bid = market.bid unit 0.05;  // the code's prices are rupees; the verifier counts ticks of
     0.05
    }
    on fill -> on_trade(trade) {
        trade.qty = fill.qty;
        trade.order_done = order.leaves == 0;
    }
    on reject -> on_reject();
    action buy -> gateway.send_buy(qty, limit unit 0.05);
}
```

Step 2 — the specification: the market and the rules that must never break

```
// clip.gts
spec ClipBuyerPython;

units { price = tick(0.05 INR); money = paise; }
ranges { price 20000 .. 40000; order_qty 1 .. 1000; }     // 1000.00 .. 2000.00 INR
assumptions { market.bid < market.ask; }

import "clip.gbind";

constraints {
    never_over:   always bought <= target;                // never buy more than the target
    clip_size:    always on new_order: order.qty <= clip;  // no order larger than a clip
    one_working:  always open_orders <= 1;                // one clip at a time
    bookkeeping:  always filled == bought;                // the strategy's count matches the exchange's
    buys_only:    always sold == 0;
}
```

`target`, `clip` and `filled` are the Python members: the specification speaks the code's language. Check what the verifier made of the code at any time with `trade-verify clip.gts --import-only`.

Step 3 — verify

```
$ trade-verify clip.gts
  VIOLATED    never_over       always bought <= target   (line 11)
              violating run, 36 events (counterexample (simulation and guided search) [slot model])
  PROVEN      clip_size        always on new_order: order.qty <= clip   (line 12)
  VIOLATED    one_working      always open_orders <= 1   (line 13)
              violating run, 4 events (counterexample (simulation and guided search) [slot model]):
             0  initial state
             1  MarketUpdate bid=20008 ask=20014 last=20014 volume=0
                  -> NewOrder O1 BUY 100 @ 20008
             2  Ack O1
             3  PartialFill O1 5 @ 20008
                  bought=5  filled=5  open_qty(buy)=95  position=5  working=0
             4  MarketUpdate bid=28018 ask=39999 last=20001 volume=100
                  -> NewOrder O2 BUY 100 @ 28018
                  open_orders=2  open_qty(buy)=195  working=1
              the constraint fails in the state after the last event
  PROVEN      bookkeeping      always filled == bought   (line 14)
  PROVEN      buys_only        always sold == 0   (line 15)
Result: CONSTRAINT VIOLATIONS FOUND (4 proven, 2 violated) in 4.79 s
```

The four-event run is the bug: **a partial fill sets `working = False`**, so the next tick sends a new clip while 95 shares of the first are still working. Repeated, it overshoots the target (the 36-event run of `never_over`). No back-test on a day without partial fills would have shown it.

Step 4 — fix and verify again

```
    def on_trade(self, trade):
        self.filled += trade.qty
        if trade.order_done:
            self.working = False
```

```
$ trade-verify clip_fixed.gts
  PROVEN      never_over       always bought <= target   (line 11)
              inductive lemmas [aggregate model: any number of orders], 0.57 s
  PROVEN      clip_size        ...
  PROVEN      one_working      ...
  PROVEN      bookkeeping      ...
  PROVEN      buys_only        ...
Result: ALL CONSTRAINTS PROVEN (6 proven) in 0.78 s
```

"Any number of orders" means the proof is not a bounded search: it covers every run, of any length.

What the importer supports

The binding and the importer are documented in Reference Manual §20. In short:

- **C++** (parsed with Clang): handler methods, member state, `if`/`else`, early returns, constant-bound loops, `double` arithmetic exactly as IEEE binary64, pointer members, unsigned wraparound, pre-trade check functions (`risk_control F(...) { rejects when returns false; }`).
- **Python** (parsed with Python's own `ast` module): handlers, `__init__` configuration (also a `setup` that runs `__init__` on your deployment's values), dicts of constant keys, strings and enumerations, lists of known length, methods inlined (also methods with several returns), `continue`, `isinstance`, `math.isclose`, `Decimal` (as binary64, noted in the output).
- **Java** (parsed with javac's tree API): the same subset with Java's integer semantics.
- Code the importer does not support **stops the import with the file and line**. It is never approximated silently.

5. Worked example B: a strategy written in GTSL

When the code is not available — or before it exists — write the rules in GTSL. Start from the pattern library:

```
trade-verify --templates                      # market_maker, twap, vwap, pov, pairs, lead_lag,
    delta_hedge, ...
trade-verify --template=twap > my_twap.gts
```

A strategy section has parameters, state and rules; rules react to events and send orders:

```
strategy Accumulate {
    parameters { initial_position = 900; max_position = 1000; }
    rules {
        on market_update { buy(100, limit = market.bid); }
    }
}
risk_controls {
    position_cap: reject when position + order.qty > max_position;
}
constraints {
    position_limit: always position <= max_position;
}
```

```
$ trade-verify examples/01_position_limit.gts
  VIOLATED    position_limit  always position <= max_position
              1  MarketUpdate ...  -> NewOrder O1 BUY 100
              2  Ack O1
              3  MarketUpdate ...  -> NewOrder O2 BUY 100
              4  Ack O2
              5  Fill O2 100 @ 20000
              6  Fill O1 100 @ 20000
```

The control looks at the position only: two orders sent before either fills both pass. The fix counts the orders in flight — `reject when position + open_qty(buy) + order.qty > max_position` — and the constraint is PROVEN.

6. Worked example C: your risk controls, against any strategy

Pre-trade risk controls must hold whatever the algorithm above them does. `mode controls_only;` replaces the rules by **any strategy**: at every step any order of any size and price, or any cancel.

```
spec DeskControls;
units  { price = tick(0.05 INR); money = paise; }
ranges { price 20000 .. 40000; order_qty 1 .. 10000; }
mode controls_only;

strategy Any { parameters { max_long = 1000; max_short = 1000; max_clip = 500; } }
```

```
risk_controls {
    clip:      reject when order.qty > max_clip;
    long_cap:  reject when order.side == buy  and position + open_qty(buy)  + order.qty > max_long;
    short_cap: reject when order.side == sell and position - open_qty(sell) - order.qty < -max_short;
    kill:      reject when kill_switch;
}

constraints {
    long_limit:  always position <= max_long;
    short_limit: always position >= -max_short;
    order_size:  always on new_order: order.qty <= max_clip;
    kill_stops:  after kill_switch: never new_order;
}
```

A PROVEN result certifies the controls for every algorithm the desk may ever run. If the controls live in C++, import them (`risk_control PreTradeRisk::allow(o, a) { ... }` in the binding, Reference Manual §20) and verify the code you deploy.

7. Writing good constraints

A constraint is a promise the strategy must keep in every situation. Good ones are short, numeric and about the trading state.

Family	Typical constraints
Exposure	`always position <= max_long`, `always abs(position) <= limit`, `always gross_exposure <= 4000000 INR`
Orders	`always on new_order: order.qty <= clip`, `always open_orders <= 1`, `always on new_order: notional(order) <= 500000 INR`
Prices	`always on new_order where order.side == buy: order.price <= market.ask` (never cross), price bands
Execution	`always bought <= quantity` (never overfill), `feasible at end: bought == quantity` (can complete)
Safety switches	`after kill_switch: never new_order`, `for each order o: after cancelled(o): never fill(o)`
Bookkeeping	`always position == initial_position + bought - sold`, `always filled == bought` (the code's own counters)
Legs and hedges	`always abs(3 * position[A] + 2 * position[B]) <= 300` (legging risk)

Tips:

- **Write the constraint you would put in a risk policy**, not one about the implementation.
- **Name it** after the policy item; the name appears in the certificate and the CI report.
- **One idea per constraint**: verdicts and runs are per constraint.
- **Add a `feasible` target** for what the strategy must be able to do. A strategy that never trades satisfies every safety constraint; the target and the automatic sanity checks catch that.

8. Writing honest assumptions

Every proof is "under these assumptions", so each assumption should describe something you know about the world:

Write	Because
`ranges { price 20000 .. 40000; order_qty 1 .. 500; }`	the prices and sizes the instrument and the venue allow
`market.bid < market.ask;`	the venue's book is never crossed
`session 09:15 .. 15:30;`	trading hours
`protocol fix44.limit_orders { late_fills; }`	the venue may fill after it acknowledged a cancel
`protocol fix44.limit_orders { fill_reports; }`	the broker sends the trade and the order status as separate messages (CTP and similar APIs)

Do **not** write an assumption to make a violation go away ("fills never come after cancels") unless the venue guarantees it — a proof under a false assumption proves nothing. The verifier helps: it checks that the assumptions still allow trading (the *sanity checks*), and conformance replay (section 12) flags every production message an assumption excludes.

9. Reading the results

```
  PROVEN      long_limit   always position <= max_long   (line 17)
              inductive lemmas [aggregate model: any number of orders], 0.03 s
```

- The **method** says how the verdict was reached: *inductive lemmas* or *invariant* for proofs, *simulation and guided search* for runs; *aggregate model* means for any number of orders, *slot model* means with the exact order lifecycle (every run it reports is real).
- A **run** lists one event per line (`MarketUpdate`, `Ack`, `PartialFill`, `Fill`, `CancelAck`, `KillSwitch`, …), the strategy's actions after `->`, the state that changed, and the order table.
- **Sanity checks** run with every verification: the assumptions allow at least one run, every rule can fire, every risk control can reject something. A failed sanity check is printed as a warning — act on it.
- **Exit codes**: 0 all proven, 1 violated, 2 undecided, 3 input error, 4 license.

10. When a constraint stays UNDECIDED

UNDECIDED is not a verdict about your strategy: the engines did not finish within the budget. In order of what helps most:

1. **Give it time**: `--timeout=300`. Proofs of strategies with real-number state take longer.
2. **Verify it alone**: `--constraint=NAME` builds the model for that constraint only.
3. **Declare ranges** for real-valued state (`state { ema: real in [0.0, 5000.0] = 0.0; }` or `range m 0.0 .. 5000.0;` in the binding). The range is checked too, so it costs nothing in trust.
4. **Split a compound constraint** into its parts.
5. **State the intermediate fact** with `assert NAME: COND;` in a rule (or a `contract` on a method in the binding): it is proved once and then helps every other proof.
6. Properties that need a product of two changing quantities (exposure × price ÷ price) can stay out of reach of the provers; the search still looks for violations, and an UNDECIDED with no violation after a long run is useful information — but it is not a proof.

11. From a result to evidence: headroom, certificates, FIX runs

```
trade-verify desk.gts --headroom
trade-verify desk.gts --report=bundle --issued-to="Example Broking Pvt Ltd"
trade-verify desk.gts --traces=runs
```

- **Headroom**: for every proven limit, the worst value any run reaches and how much of the limit is never used, rated 1–5 stars (the Giolit Risk Resilience Score).

- **Report bundle**: `certificate.pdf` (all proven), `violation-report.pdf` or `verification-report.pdf`; the signed statement `certificate.json`; the proofs in your specification's terms (`proof.json`); every run as text, JSON, FIX 4.4 messages and a CSV market path; `verify_certificate.py` with which anyone can check the bundle offline.
- **Runs as FIX logs**: replay a violating run in your own test harness or simulator to reproduce the bug there.

12. Keeping production honest: conformance replay

A proof covers the model. Conformance replay checks that production stays inside the model, using the logs you already have:

```
$ trade-verify desk.gts --conform=dropcopy-2026-10-09.fix
  IMPOSSIBLE message 7 (line 10): a fill needs a live acknowledged order (New, PartFilled or PendingCancel)
  CONTROLS   message 8 (line 11): the risk controls of the specification reject this order, but it was sent
      to the venue
Result: THE LOG DOES NOT CONFORM TO THE SPECIFICATION
```

`IMPOSSIBLE` means an assumption is wrong (change the specification and verify again); `UNEXPECTED` or `CONTROLS` means production did something the verified design does not do. Run it nightly on drop copies, or live with `--follow`.

13. Verification as a release gate

```
trade-verify strategy.gts --timeout=120 --json=results.json --junit=results.xml --baseline=release-1.4.json
```

- `--junit` gives one test case per constraint for GitHub, GitLab or Jenkins (a violation fails with its run).
- `--baseline` compares with the last release: a constraint that was proven and no longer is, is a regression (exit code 5).
- `share/giolit/ci/` in the installation has a GitHub Actions workflow and a GitLab CI job.

14. A checklist for every strategy

1. The constraints are the desk's risk policy, named after it.
2. The ranges and assumptions describe the venue and the instrument; none of them hides a violation.
3. At least one `feasible` target shows the strategy can do its job.
4. Every constraint is PROVEN (or every violation is understood and accepted in writing).
5. No sanity-check warnings.
6. The certificate bundle is archived with the release.
7. Conformance replay runs on the production drop copy.
8. The specification is in version control next to the strategy, and the CI gate runs on every change.

Giolit Labs, Thrissur, Kerala, India — www.giolit.com — business.giolitlabs@gmail.com